

A LECTURE ON

Harness, Context, Evals.

Engineering systems around model intelligence.

Three sides of the same problem —

what wraps the model · what goes in the window · how we know it works

Today's arc.

- | | | |
|----|----------------|--------------------------------------|
| 01 | Harness | <i>what wraps the model</i> |
| 02 | Context | <i>what goes in the window</i> |
| 03 | Evals | <i>training data for the harness</i> |

Three sides of the same problem.

Same model. Three harnesses.

Why don't they get the same score?

HARNESS	SCORE	MODEL
ForgeCode	79.8%	<i>Claude Opus 4.6</i>
Capy	75.3%	<i>Claude Opus 4.6</i>
Claude Code	lower	<i>Claude Opus 4.6</i>

Terminal-Bench 2.0 — same model, different harness.

BEAT 1

Harness

Agent equals model plus harness.

If you're not the model, you're the harness.

Agent = Model + Harness

Harness primitives.

Each one patches a specific model deficiency.

- | | | |
|----|--------------------|---|
| 01 | Filesystem | — <i>no durable state</i> |
| 02 | Bash + code | — <i>no general-purpose action</i> |
| 03 | Sandbox | — <i>unsafe execution at scale</i> |
| 04 | Sub-agents | — <i>context contamination</i> |
| 05 | Middleware | — <i>no deterministic interventions</i> |

Model-harness-fit.

The harness is part of the model's effective parameters.

CODEX / OPENAI

CLAUDE / ANTHROPIC

`apply_patch (diff)`

`Edit (old/new string)`

`<oai-mem-citation>`

`<system-reminder>`

`exec_command_tool`

`Bash + Monitor`

Same operations. Different vocabularies. Baked in by post-training.

Three consequences.

01 **No model-agnostic agent.**

Honest version — per-model harness; you pick a product, not a model.

02 **Mid-chat model swaps break.**

Transcript OOD, cache miss, tool-surface mismatch — all at once.

03 **The matched pair shifts.**

Yesterday's load-bearing scaffold is today's dead weight.

Beat 1.

- *Primitives are derived, not invented.*
- *Match the pair: harness $\leftrightarrow$ model.*
- *Scaffolding goes stale — delete code.*

Add primitives. Watch failures fix.

- 01 **Tool-result clearing** — *fix output bloat*
- 02 **Loop detection** — *break out of doom loops*
- 03 **Sub-agent dispatch** — *protect parent context*

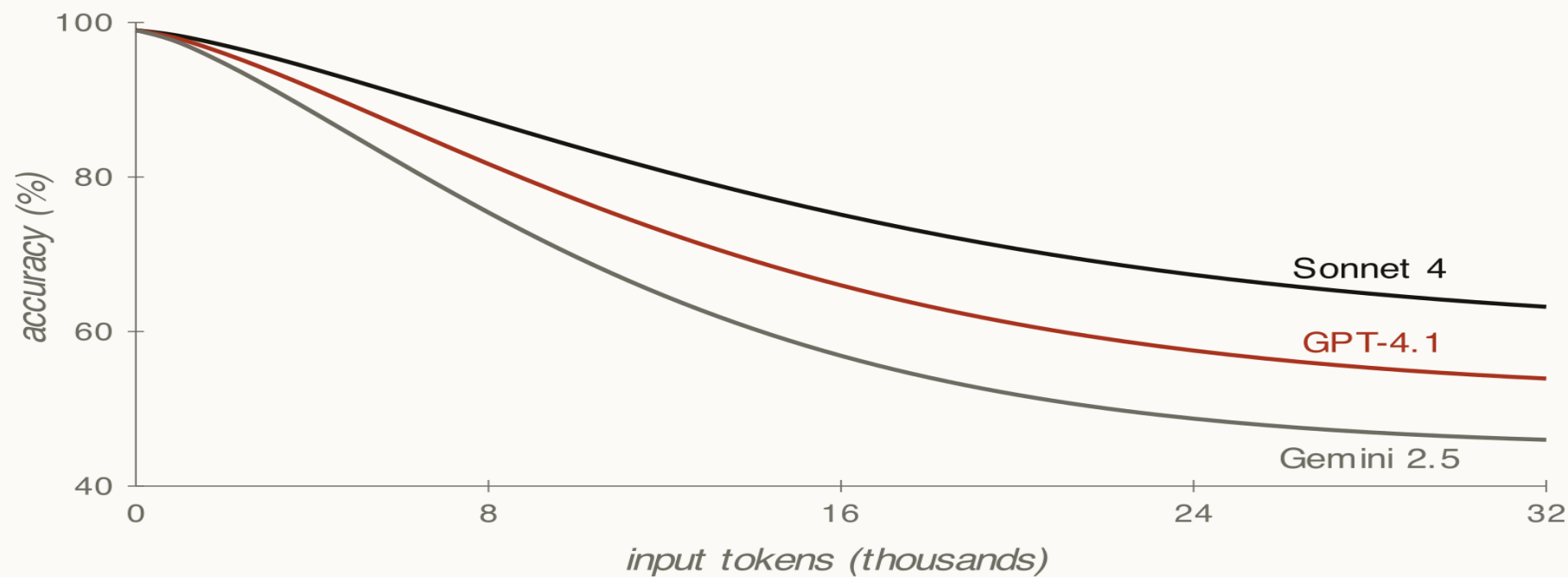
BEAT 2

Context

Context rot.

Context rot — accuracy declines with input length

Chroma research, 2025 · trivial copy task



Frontier models break on a trivial copy task as input grows.

Attention budget.

Context is finite — in performance, not just tokens.

**n tokens $\rightarrow$ n^2 pairwise
relationships**

Every token costs attention budget.

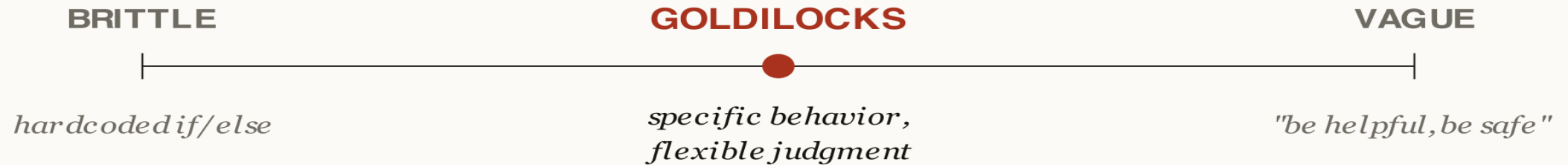
THE PRINCIPLE

Smallest set of high-signal tokens.

Right altitude.

Right altitude.

System prompts: not too rigid, not too vague.



Specific behavior, flexible judgment.

Pre-loaded versus just-in-time.

PRE-LOADED / RAG

Stuff context up front

Fast on small data

Drowns on large data

Throws away metadata

JUST-IN-TIME / CLAUDE CODE

Lightweight identifiers

Agent navigates via tools

Progressive disclosure

Mirrors human cognition

Long-horizon strategies.

STRATEGY

USE WHEN

EXAMPLE

Compaction*conversational flow*

summary-then-continue

Note-taking*iterative milestones*

NOTES.md, scratchpad

Sub-agents*parallel exploration*

clean context per branch

Reproduce rot. Then fix it.

- 01 **Reproduce context rot live** — *Chroma's repeat-words task*
- 02 **Compare pre-load vs JIT** — *on a research task*
- 03 **Add compaction** — *watch the saw-tooth*

BEAT 3

Evals

Why evals.

Evals are training data
for harness improvement.

harness + evals + harness engineering → better agent

Same loop as supervised learning. Gradient flows into the harness.

Vocabulary.

task	<i>single test, defined inputs + success criteria</i>
-------------	---

trial	<i>one attempt at a task (run multiple)</i>
--------------	---

grader	<i>scoring logic — code, LLM, or human</i>
---------------	--

transcript	<i>full record — outputs, tool calls, reasoning</i>
-------------------	---

outcome	<i>final state in the environment</i>
----------------	---------------------------------------

eval harness vs agent harness	<i>different layers — don't conflate</i>
--------------------------------------	--

The mirage of generic metrics.

MMLU, HELM, BERTScore — these don't tell you if your product works.

FOUNDATION EVAL

Is the model generally capable?

Standardized

Cross-model

Easy to game

Weak signal for products

PRODUCT EVAL

Does YOUR pipeline do its job?

Domain-specific

Failure-mode-driven

Custom criteria

Strong signal for shipping

The 5-star lie.

What does a 3.7 in "helpfulness" mean?

Binary > Likert

Forces clarity

Faster to label

More consistent

Actionable

Decompose nuance into multiple binary checks, not a fuzzier scale.

Critique shadowing.

Hamel Husain's process for building product evals.

- 01 Find THE principal domain expert
- 02 Build diverse dataset (features × scenarios × personas)
- 03 Pass / fail + detailed critique
- 04 Fix obvious agent errors
- 05 Build LLM judge, iterate to alignment
- 06 Error analysis at scale

LLM judge alignment.

Different splits than ML — small train, big test.



Measure TPR + TNR — not raw agreement.

pass@k versus pass^k.

Two metrics. Opposite stories.

pass@k

Any path to success in k tries?

↑ with k

first-try problems

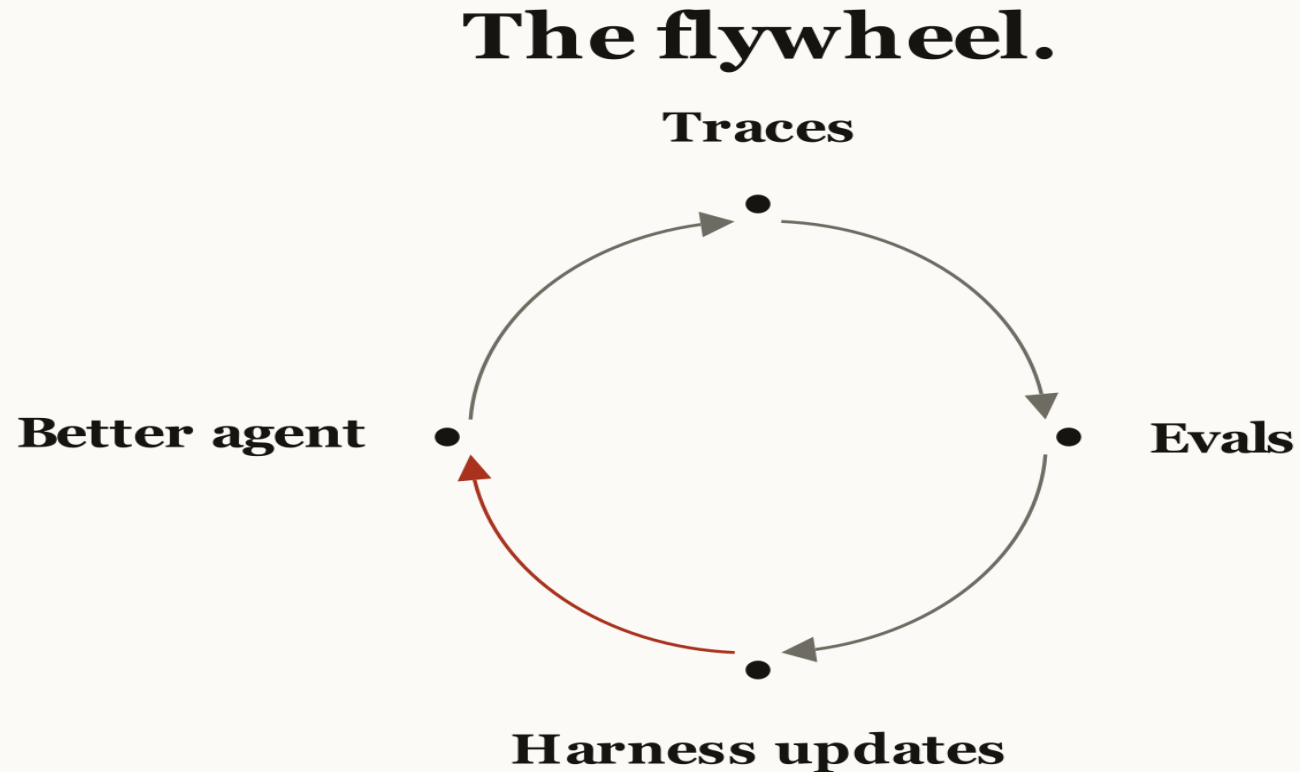
pass^k

All k trials succeed?

↓ with k

reliability-critical

The flywheel.



Every trace is a potential eval. Every eval improves the harness.

Build an aligned LLM judge.

- 01 Synthesize 20 diverse queries for company-research agent
- 02 Hand-label pass / fail with critiques
- 03 Train / dev / test split — write judge v1 (no few-shot)
- 04 Iterate to v2 with few-shot, lock and test
- 05 Cluster failure modes → harness backlog

Three takeaways.

01 **Agent = Model + Harness.**

harness is part of the model's effective parameters

02 **Context is finite — in performance.**

smallest set of high-signal tokens, JIT > pre-load

03 **Evals are training data.**

binary > Likert, aligned LLM judge, the flywheel never ends